

Frontend Module

Full-Stack Development

HTML: Structuring Content

HTML (HyperText Markup Language) is the skeleton of every web page — it doesn't describe how things look, only what they are. Writing semantic HTML means choosing tags that describe the actual meaning of content, not just its visual appearance: a navigation bar goes in a `<nav>`, a self-contained article goes in `<article>`, a heading uses `<h1>`-`<h6>` in order, not just whichever looks the right size. Semantic markup matters for three concrete reasons: screen readers rely on it to describe a page to visually impaired users, search engines use it to understand page structure, and it makes the DOM easier for other developers (and your own future self) to reason about.

CSS Fundamentals

CSS (Cascading Style Sheets) controls layout, color, spacing, and typography. Three ideas explain almost all CSS behavior in practice:

- The box model — every element is a rectangular box made of content, padding, border, and margin, in that order from inside out. Most layout bugs come from misunderstanding how these stack.
- Specificity and the cascade — when multiple rules target the same element, the browser picks a winner based on selector specificity (IDs beat classes, classes beat element selectors) and source order (later rules win ties).
- Inheritance — some properties (like font and color) automatically pass down from parent to child elements unless overridden; others (like border and padding) don't.

Modern Layout: Flexbox and Grid

Flexbox lays elements out along a single axis (row or column) and excels at distributing space and aligning items — think navigation bars, button groups, and centering content. CSS Grid lays elements out on a full two-dimensional grid of rows and columns at once, and is the better tool whenever a layout needs to align things across both axes simultaneously — think page layouts, card grids, and dashboards. In practice, most real interfaces use both together: Grid for the overall page structure, Flexbox for the smaller components inside each grid area.

Responsive Design

A responsive page adapts its layout to the screen it's rendered on rather than assuming one fixed size. The two core techniques are fluid sizing (using relative units like %, rem, and viewport units instead of fixed pixels so elements scale naturally) and media queries (CSS rules that apply only above or below a certain screen width, letting you change layout — e.g. switching a 3-column grid to a single column — at defined breakpoints). The standard approach today is mobile-first: write the base styles for the smallest screen, then use media queries to add complexity as the screen gets larger, rather than the reverse.

JavaScript & the DOM

The Document Object Model (DOM) is the browser's live, in-memory tree representation of the HTML page — JavaScript reads and modifies this tree to make pages interactive after they've loaded. Core DOM skills include selecting elements (`document.querySelector`), reading and changing their content or attributes, and attaching event listeners (`addEventListener`) that run code in response to user actions like clicks, form submissions, or key presses. Every interactive behavior on a page — a dropdown opening, a form validating itself, a button changing color on click — ultimately comes down to this same pattern: select an element, listen for an event, update the DOM in response.

React Component Architecture

React structures a UI as a tree of components — small, reusable functions that each return the markup for one piece of the interface, composed together to build the full page. Two concepts drive almost everything else in React:

- Props — data passed down from a parent component to a child, making the child reusable and configurable rather than hard-coded.
- State — data a component owns and can change itself over time (via `useState`); when state changes, React automatically re-renders that component (and its children) to reflect the new value.
- Unidirectional data flow — data flows down through props, and changes flow back up through callback functions passed as props, keeping the app's data predictable even as it grows.

State Management

As an app grows past a handful of components, keeping state organized becomes its own skill. Local component state (`useState`) is enough when only one component and its direct children need a value. When distant, unrelated components need to share and react to the same data — like a logged-in user or shopping cart — that state needs to move up to a common ancestor ('lifting state up') or into a dedicated store (React Context for simpler cases, or a library like Redux/Zustand for larger apps) so every component that needs it can read and update it consistently.

Learn More

MDN Web Docs — HTML, CSS & JavaScript Reference

<https://developer.mozilla.org/en-US/docs/Web>

React Official Documentation

<https://react.dev/>

freeCodeCamp YouTube Channel

<https://www.youtube.com/@freecodecamp>